RAPPORT DE TRAVAUX PRATIQUES

Série d'exercices — Perceptron / Deep Learning

IDENTITÉ

Nom :	Jalal Grini
Classe :	TDIA2
École :	ENSAH
Enseignant :	M.Cherradi
Sujet :	Perceptron – Série d'exercices

Table des matières

1 Exercice : Classification des portes logiques AND et OR **2**

2 Exercice : Classification binaire sur les données Iris Setosa **13**

3 Exercice : Régression linéaire avec le Perceptron **19**

4 Exercice : Classification sur MNIST (digits) **24**

5 Exercice : Segmentation des données synthétiques (make_blobs) **29**

Exercice : Classification des portes logiques AND et OR

Énoncé

Apprendre au Perceptron les fonctions logiques AND et OR en se conformant aux étapes suivantes :

1. Créez les ensembles de données pour les portes logiques AND et OR.
2. Entraînez un modèle Perceptron sur les données pour chaque porte séparément.
3. Testez le modèle sur des données de validation.
4. Commentez la capacité du Perceptron à apprendre les deux types de portes.

1) Génération des données

ex1.py

```python
# Donn es communes    toutes les impl mentations
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y_or = np.array([0, 1, 1, 1])
y_and = np.array([0, 0, 0, 1])
```

Explication Le bloc ci-dessus crée le jeu de données d'entraînement pour les portes logiques (toutes les combinaisons binaires de deux bits). `X` contient les entrées ; `y_or` et `y_and` sont les labels pour OR et AND respectivement.

2) Fonction d'activation

ex1.py

```python
def fct_activattion(c):
    """Fonction d'activation seuil (step function)"""
    if c >= 0.1:
        return 1
    else:
        return 0

def step_activation(x):
    """Step function for perceptron (vectorized)"""
    return (x >= 0).astype(int)
```

Explication La fonction d'activation est un seuil (step function) fixé à 0.1. Elle renvoie 1 si la somme pondérée est supérieure ou égale au seuil, sinon 0. Nous utilisons également une version vectorisée de cette fonction pour l'implémentation Keras.

3) Entraînement (from scratch)

ex1.py

```python
def training(x, y, learn_rate=0.1, epochs=5):
    n = x.shape[1]
    w = np.zeros(n)
    b = 0
    for epoch in range(epochs):
        for i in range(len(x)):
            c = np.dot(x[i], w) + b
            y_pre = fct_activattion(c)
            err = y[i] - y_pre
            w += learn_rate * err * x[i]
            b += learn_rate * err
    return w, b

def prediction(x, w, b):
    prediction = []
    for i in range(len(x)):
        z = np.dot(x[i], w) + b
        prediction.append(fct_activattion(z))
    return np.array(prediction)
```

Explication L'algorithme met en œuvre la règle du Perceptron : on initialise les poids à zéro, puis pour chaque exemple et chaque époque on calcule l'erreur et on met à jour les poids et le biais.

4) Prédiction et évaluation (from scratch)

ex1.py

```python
print("test for AND:\n")
w_and, b_and = training(X, y_and)
and_pre = prediction(X, w_and, b_and)
print("the weight is :", w_and, " and the bias is :", b_and)
print("the AND prediction is : ", and_pre)
print("the real AND is : ", y_and)

print("\ntest for OR:\n")
w_or, b_or = training(X, y_or)
or_pre = prediction(X, w_or, b_or)
print("the weight is :", w_or, " and the bias is :", b_or)
print("the OR prediction is : ", or_pre)
print("the real OR is : ", y_or)
```

Explication La fonction `prediction` applique la combinaison linéaire puis la fonction d'activation pour retourner les prédictions. Le script teste ensuite AND puis OR (entraînement puis prédiction sur le même jeu).

5) Implémentation avec scikit-learn

ex1.py

```python
def sk_train(X, y):
    clf = perceptron.Perceptron(eta0=0.1, max_iter=10, shuffle=False)
    clf.fit(X, y)
    prediction = clf.predict(X)
    return clf, prediction
```

Explication La classe `Perceptron` de `sklearn.linear_model` encapsule l'entraînement et fournit les poids via `clf.coef_` et le biais via `clf.intercept_`.

6) Résultats et comparaison avec scikit-learn

ex1.py

```python
# Comparison between from-scratch and sklearn on TRAINING data
print("\n*** COMPARISON: From Scratch vs Sklearn (on TRAINING data) ***")
print("From scratch AND predictions:", and_pre)
print("Sklearn AND predictions:     ", and_sk_pre)
print("Match:", np.array_equal(and_pre, and_sk_pre))
print()
print("From scratch OR predictions: ", or_pre)
print("Sklearn OR predictions:      ", or_sk_pre)
print("Match:", np.array_equal(or_pre, or_sk_pre))

# VALIDATION ON SEPARATE DATA
print("\n" + "="*60)
print("VALIDATION ON NEW/TEST DATA")
print("="*60)

# Using the same data for validation (in this case, XOR-like validation
    ↪ to test limits)
# Same as training (complete dataset)
X_test = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])

print("\n--- VALIDATION: And Gate ---")
and_pre_test = prediction(X_test, w_and, b_and)
print("AND Training Predictions: ", and_pre)
print("AND Validation Predictions:", and_pre_test)
print("Accuracy (Training): {:.2%}".format(np.mean(and_pre == y_and)))
print("Accuracy (Validation): {:.2%}".format(np.mean(and_pre_test ==
    ↪ y_and)))

print("\n--- VALIDATION: OR Gate ---")
or_pre_test = prediction(X_test, w_or, b_or)
print("OR Training Predictions: ", or_pre)
print("OR Validation Predictions:", or_pre_test)
print("Accuracy (Training): {:.2%}".format(np.mean(or_pre == y_or)))
print("Accuracy (Validation): {:.2%}".format(np.mean(or_pre_test == y_or)
    ↪ ))
```

Explication Nous comparons maintenant les résultats de l'implémentation from scratch avec ceux de scikit-learn sur les données d'entraînement. Nous validons également nos modèles sur un jeu de données séparé (dans ce cas, nous utilisons le même jeu de données pour simplifier l'exemple). Les deux implémentations donnent des résultats identiques, ce qui confirme la correction de notre implémentation from scratch.

2) Fonction d'activation

ex1.py

```python
def fct_activattion(c):
    """Fonction d'activation seuil (step function)"""
    if c >= 0.1:
        return 1
    else:
        return 0

def step_activation(x):
    """Step function for perceptron (vectorized)"""
    return (x >= 0).astype(int)
```

Explication La fonction d'activation est un seuil (step function) fixé à 0.1. Elle renvoie 1 si la somme pondérée est supérieure ou égale au seuil, sinon 0. Nous utilisons également une version vectorisée de cette fonction pour l'implémentation Keras.

3) Entraînement (from scratch)

ex1.py

```python
def training(x, y, learn_rate=0.1, epochs=5):
    n = x.shape[1]
    w = np.zeros(n)
    b = 0
    for epoch in range(epochs):
        for i in range(len(x)):
            c = np.dot(x[i], w) + b
            y_pre = fct_activattion(c)
            err = y[i] - y_pre
            w += learn_rate * err * x[i]
            b += learn_rate * err
    return w, b

def prediction(x, w, b):
    prediction = []
    for i in range(len(x)):
        z = np.dot(x[i], w) + b
        prediction.append(fct_activattion(z))
    return np.array(prediction)
```

Explication L'algorithme met en œuvre la règle du Perceptron : on initialise les poids à zéro, puis pour chaque exemple et chaque époque on calcule l'erreur et on met à jour

les poids et le biais.

4) Prédiction et évaluation (from scratch)

ex1.py

```python
print("test for AND:\n")
w_and, b_and = training(X, y_and)
and_pre = prediction(X, w_and, b_and)
print("the weight is :", w_and, " and the bias is :", b_and)
print("the AND prediction is : ", and_pre)
print("the real AND is : ", y_and)

print("\ntest for OR:\n")
w_or, b_or = training(X, y_or)
or_pre = prediction(X, w_or, b_or)
print("the weight is :", w_or, " and the bias is :", b_or)
print("the OR prediction is : ", or_pre)
print("the real OR is : ", y_or)
```

Explication La fonction `prediction` applique la combinaison linéaire puis la fonction d'activation pour retourner les prédictions. Le script teste ensuite AND puis OR (entraînement puis prédiction sur le même jeu).

5) Implémentation avec scikit-learn

ex1.py

```python
def sk_train(X, y):
    clf = perceptron.Perceptron(eta0=0.1, max_iter=10, shuffle=False)
    clf.fit(X, y)
    prediction = clf.predict(X)
    return clf, prediction
```

Explication La classe `Perceptron` de `sklearn.linear_model` encapsule l'entraînement et fournit les poids via `clf.coef_` et le biais via `clf.intercept_`.

6) Résultats et comparaison avec scikit-learn

ex1.py

```python
# Comparison between from-scratch and sklearn on TRAINING data
print("\n*** COMPARISON: From Scratch vs Sklearn (on TRAINING data) ***")
print("From scratch AND predictions:", and_pre)
print("Sklearn AND predictions:     ", and_sk_pre)
print("Match:", np.array_equal(and_pre, and_sk_pre))
print()
print("From scratch OR predictions: ", or_pre)
print("Sklearn OR predictions:      ", or_sk_pre)
print("Match:", np.array_equal(or_pre, or_sk_pre))
```

```python

# VALIDATION ON SEPARATE DATA
print("\n" + "="*60)
print("VALIDATION ON NEW/TEST DATA")
print("="*60)

# Using the same data for validation (in this case, XOR-like validation to test limits)
# Same as training (complete dataset)
X_test = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])

print("\n--- VALIDATION: AND Gate ---")
and_pre_test = prediction(X_test, w_and, b_and)
print("AND Training Predictions: ", and_pre)
print("AND Validation Predictions:", and_pre_test)
print("Accuracy (Training): {:.2%}".format(np.mean(and_pre == y_and)))
print("Accuracy (Validation): {:.2%}".format(np.mean(and_pre_test == y_and)))

print("\n--- VALIDATION: OR Gate ---")
or_pre_test = prediction(X_test, w_or, b_or)
print("OR Training Predictions: ", or_pre)
print("OR Validation Predictions:", or_pre_test)
print("Accuracy (Training): {:.2%}".format(np.mean(or_pre == y_or)))
print("Accuracy (Validation): {:.2%}".format(np.mean(or_pre_test == y_or) ))
```

Explication Nous comparons maintenant les résultats de l'implémentation from scratch avec ceux de scikit-learn sur les données d'entraînement. Nous validons également nos modèles sur un jeu de données séparé (dans ce cas, nous utilisons le même jeu de données pour simplifier l'exemple). Les deux implémentations donnent des résultats identiques, ce qui confirme la correction de notre implémentation from scratch.

7) Implémentation avec Keras

ex1.py

```python
# Create a simple perceptron-like model using Keras
def create_perceptron_model():
    model = Sequential([
        Dense(1, input_shape=(2,), activation='linear',
              kernel_initializer='zeros', bias_initializer='zeros')
    ])
    return model

# Train using perceptron learning rule (manual implementation with Keras model)
def train_perceptron(model, X, y, learning_rate=0.1, epochs=5):
    # Get weights and bias
    weights = model.get_weights()[0]  # weight matrix (2, 1)
    bias = model.get_weights()[1]     # bias vector (1,)

    # Perceptron learning rule
```

```python
    for epoch in range(epochs):
        for i in range(len(X)):
            # Linear combination
            linear_output = np.dot(X[i], weights[:, 0]) + bias[0]  # Extract column 0
            # Step activation
            y_pred = step_activation(linear_output)
            # Error
            error = y[i] - y_pred
            # Update if error exists
            if error != 0:
                weights[:, 0] += learning_rate * error * X[i]  # Update column 0
                bias[0] += learning_rate * error

    # Update model weights
    model.set_weights([weights, bias])
    return model

def predict_perceptron(model, X):
    weights = model.get_weights()[0]  # (2, 1)
    bias = model.get_weights()[1]     # (1,)
    linear_output = np.dot(X, weights[:, 0]) + bias[0]  # Extract column 0
    return step_activation(linear_output)
```

Explication Nous définissons ensuite notre modèle perceptron en utilisant Keras. Le modèle consiste en une seule couche dense avec 2 entrées (pour les deux caractéristiques) et 1 sortie. Nous utilisons une activation linéaire car nous allons appliquer manuellement la fonction d'activation de seuil (step function) lors de l'entraînement, ce qui nous permet de suivre exactement la même règle d'apprentissage que dans l'implémentation from scratch.

ex1.py

```python
# Test AND gate
print("test for AND:\n")
model_and = create_perceptron_model()
model_and = train_perceptron(model_and, X, y_and, learning_rate=0.1, epochs=5)
and_pred = predict_perceptron(model_and, X)
weights_and = model_and.get_weights()[0][:, 0]  # Extract column 0 and flatten
bias_and = model_and.get_weights()[1][0]
print("the weight is :", weights_and, " and the bias is :", bias_and)
print("the AND prediction is : ", and_pred)
print("the real AND is : ", y_and)

print("\n----------------------------------------")
print("test for OR:\n")
model_or = create_perceptron_model()
model_or = train_perceptron(model_or, X, y_or, learning_rate=0.1, epochs=5)
or_pred = predict_perceptron(model_or, X)
```

```python
weights_or = model_or.get_weights()[0][:, 0]  # Extract column 0 and flatten
bias_or = model_or.get_weights()[1][0]
print("the weight is :", weights_or, " and the bias is :", bias_or)
print("the OR prediction is : ", or_pred)
print("the real OR is : ", y_or)

print("\n----------------------------------------")
print("VALIDATION ON SAME DATA (for completeness)")
print("AND Validation Predictions:", predict_perceptron(model_and, X))
print("OR Validation Predictions:", predict_perceptron(model_or, X))
print("Accuracy (AND): {:.2%}".format(np.mean(predict_perceptron(model_and, X) == y_and)))
print("Accuracy (OR): {:.2%}".format(np.mean(predict_perceptron(model_or, X) == y_or)))
```

Explication Finalement, nous testons notre modèle sur les portes AND et OR, tout comme dans les implémentations précédentes. Nous affichons les poids et biais appris, ainsi que les prédictions du modèle comparées aux valeurs réelles. Nous calculons également la précision sur les mêmes données d'entraînement (validation) pour compléter l'analyse.

6) Implémentation avec Keras

ex1.py

```python
print("------------------keras method----------------")

X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y_or = np.array([0, 1, 1, 1])
y_and = np.array([0, 0, 0, 1])
```

Explication Tout comme dans les implémentations précédentes, nous commençons par créer les ensembles de données pour les portes logiques AND et OR. `X` contient les entrées ; `y_or` et `y_and` sont les labels pour OR et AND respectivement.

ex1.py

```python
def step_activation(x):
    """Step function for perceptron"""
    return (x >= 0).astype(int)

# Create a simple perceptron-like model using Keras
def create_perceptron_model():
    model = Sequential([
        Dense(1, input_dim=2, activation='linear',
              kernel_initializer='zeros', bias_initializer='zeros')
    ])
    return model
```

Explication Nous définons ensuite notre modèle perceptron en utilisant Keras. Le modèle consiste en une seule couche dense avec 2 entrées (pour les deux caractéristiques) et 1 sortie. Nous utilisons une activation linéaire car nous allons appliquer manuellement la fonction d'activation de seuil (step function) lors de l'entraînement, ce qui nous permet de suivre exactement la même règle d'apprentissage que dans l'implémentation from scratch.

ex1.py

```python
# Train using perceptron learning rule (manual implementation with Keras model)
def train_perceptron(model, X, y, learning_rate=0.1, epochs=5):
    # Get weights and bias
    weights = model.get_weights()[0]  # weight matrix (2, 1)
    bias = model.get_weights()[1]     # bias vector (1,)

    # Perceptron learning rule
    for epoch in range(epochs):
        for i in range(len(X)):
            # Linear combination
            linear_output = np.dot(X[i], weights[:, 0]) + bias[0]  # Extract column 0
            # Step activation
            y_pred = step_activation(linear_output)
            # Error
            error = y[i] - y_pred
            # Update if error exists
            if error != 0:
                weights[:, 0] += learning_rate * error * X[i]  # Update column 0
                bias[0] += learning_rate * error

    # Update model weights
    model.set_weights([weights, bias])
    return model

def predict_perceptron(model, X):
    weights = model.get_weights()[0]  # (2, 1)
    bias = model.get_weights()[1]     # (1,)
    linear_output = np.dot(X, weights[:, 0]) + bias[0]  # Extract column 0
    return step_activation(linear_output)
```

Explication Nous implémentons ensuite la règle d'apprentissage du perceptron de manière manuelle, tout comme dans l'implémentation from scratch. Cette fonction calcule la sortie linéaire, applique la fonction d'activation de seuil, calcule l'erreur, et met à jour les poids et le biais selon la règle du perceptron. Cette approche nous permet de comparer directement les trois méthodes en utilisant exactement le même algorithme d'apprentissage.

ex1.py

```python
# Test AND gate
print("test for AND:\n")
model_and = create_perceptron_model()
```

```python
model_and = train_perceptron(model_and, X, y_and, learning_rate=0.1, epochs=5)
and_pred = predict_perceptron(model_and, X)
weights_and = model_and.get_weights()[0][:, 0]  # Extract column 0 and flatten
bias_and = model_and.get_weights()[1][0]
print("the weight is :", weights_and, " and the bias is :", bias_and)
print("the AND prediction is : ", and_pred)
print("the real AND is : ", y_and)

print("\n----------------------------------------")
print("test for OR:\n")
model_or = create_perceptron_model()
model_or = train_perceptron(model_or, X, y_or, learning_rate=0.1, epochs=5)
or_pred = predict_perceptron(model_or, X)
weights_or = model_or.get_weights()[0][:, 0]  # Extract column 0 and flatten
bias_or = model_or.get_weights()[1][0]
print("the weight is :", weights_or, " and the bias is :", bias_or)
print("the OR prediction is : ", or_pred)
print("the real OR is : ", y_or)

print("\n----------------------------------------")
print("VALIDATION ON SAME DATA (for completeness)")
print("AND Validation Predictions:", predict_perceptron(model_and, X))
print("OR Validation Predictions:", predict_perceptron(model_or, X))
print("Accuracy (AND): {:.2%}".format(np.mean(predict_perceptron(model_and, X) == y_and)))
print("Accuracy (OR): {:.2%}".format(np.mean(predict_perceptron(model_or, X) == y_or)))
```

Explication Finalement, nous testons notre modèle sur les portes AND et OR, tout comme dans les implémentations précédentes. Nous affichons les poids et biais appris, ainsi que les prédictions du modèle comparées aux valeurs réelles. Nous calculons également la précision sur les mêmes données d'entraînement (validation) pour compléter l'analyse.

8) Analyse et conclusions

ex1.py

```python
# COMMENTARY ON PERCEPTRON LEARNING ABILITY
print("\n" + "="*60)
print("ANALYSIS: PERCEPTRON LEARNING ABILITY")
print("="*60)
print"""
FINDINGS:

1. LINEAR SEPARABILITY:
   - The Perceptron successfully learns BOTH AND and OR gates
   - These functions are LINEARLY SEPARABLE,
     meaning a single straight line can separate them
   - The Perceptron is designed for linearly separable problems
```

```

2. AND GATE:
   - Only one input combination (1,1) produces output 1
   - The hyperplane learned separates this clearly
   - The Perceptron learns this perfectly

3. OR GATE:
   - Three input combinations (0,1), (1,0), (1,1) produce output 1
   - The hyperplane separates "at least one 1" from "both 0"
   - The Perceptron learns this perfectly

4. LIMITATIONS OF PERCEPTRON:
   - The Perceptron CANNOT learn XOR (exclusive OR) gate
   - XOR is NOT linearly separable (requires multiple layers)
   - This was a fundamental limitation discovered in the 1960s
   - Multi-layer neural networks are needed for XOR

5. CONVERGENCE:
   - With proper learning rate and epochs, both gates converge
   - The from-scratch and sklearn implementations match
   - Training on the complete dataset = validation on the same data
   - (In practice, you would use train/test split for generalization testing)

CONCLUSION:
The Perceptron is effective for linearly separable logical functions like AND and OR.
For non-linearly separable functions (like XOR), deeper architectures are required.
""")
```

Résultats numériques

Les résultats imprimés par le script (exécution locale) sont les suivants :

- AND — Accuracy (Training) : 100.00% ; Accuracy (Validation) : 100.00%.
- OR — Accuracy (Training) : 100.00% ; Accuracy (Validation) : 100.00%.

Interprétation Le Perceptron apprend parfaitement les fonctions linéairement séparables (AND, OR). La faiblesse sur la validation pour AND indique que la frontière apprise n'est pas généralisable sur ce petit exemple synthétique.

Exercice : Classification binaire sur les données Iris Setosa

Énoncé

Chargez le jeu de données Iris. Sélectionnez deux caractéristiques (longueur et largeur des pétales). Entraînez le Perceptron pour distinguer Iris Setosa des autres catégories. Calculez la précision du modèle sur des données de test.

1) Chargement et sélection des caractéristiques

ex2.py

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Perceptron
from sklearn.metrics import (
    accuracy_score, confusion_matrix, classification_report
)

# Load Iris Dataset
iris = datasets.load_iris()
X    = iris.data
y    = iris.target

# Select Petal Length and Petal Width
X_selected = X[:, [2, 3]]

# Create binary labels: Setosa (1) vs Others (0)
y_binary = (y == 0).astype(int)
```

Explication On charge le dataset Iris via `sklearn.datasets` puis on sélectionne deux caractéristiques (index 2 et 3 : longueur et largeur des pétales). Les labels sont transformés en problème binaire (Setosa / Non-Setosa).

2) Prétraitement et entraînement

ex2.py

```python
# Split data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X_selected, y_binary,
    test_size=0.2, random_state=42, stratify=y_binary
)

# Standardize features
scaler         = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled  = scaler.transform(X_test)

# Train Perceptron
clf = Perceptron(eta0=0.1, max_iter=100, random_state=42, tol=1e-3)
clf.fit(X_train_scaled, y_train)
```

Explication La standardisation est importante pour la convergence du Perceptron. On utilise `StandardScaler` puis on entraîne `Perceptron` de sklearn avec un taux d'apprentissage et un nombre d'itérations fixés.

3) Évaluation et visualisation

ex2.py

```
# Make predictions
y_pred_train = clf.predict(X_train_scaled)
y_pred_test  = clf.predict(X_test_scaled)

# Evaluate model
train_accuracy = accuracy_score(y_train, y_pred_train)
test_accuracy  = accuracy_score(y_test,  y_pred_test)

print(f"Training Accuracy: {train_accuracy:.2%}")
print(f"Test Accuracy:     {test_accuracy:.2%}")

print("\nConfusion Matrix (Test Data):")
cm = confusion_matrix(y_test, y_pred_test)
print(cm)

print("\nClassification Report:")
print(classification_report(
    y_test, y_pred_test,
    target_names=['Non-Setosa', 'Setosa']
))
```

Explication Les métriques utilisées comprennent la précision et la matrice de confusion. Le script sauvegarde également une figure (`iris_classification.png`) pour visualiser la frontière de décision.

4) Implémentation avec Keras

ex2.py

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix,
    ↪ classification_report
```

```python
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import SGD

# Load Iris Dataset
iris = datasets.load_iris()
X = iris.data
y = iris.target

# Select Petal Length and Petal Width
X_selected = X[:, [2, 3]]

# Create binary labels: Setosa (1) vs Others (0)
y_binary = (y == 0).astype(int)
```

Explication Comme dans les implémentations précédentes, nous commençons par charger le dataset Iris via `sklearn.datasets` puis nous sélectionnons deux caractéristiques (index 2 et 3 : longueur et largeur des pétales). Les labels sont transformés en problème binaire (Setosa / Non-Setosa).

ex2.py

```python
# Split data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X_selected, y_binary, test_size=0.2, random_state=42, stratify=
    ↪ y_binary
)

# Standardize features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Explication La standardisation est importante pour la convergence du Perceptron. Nous utilisons `StandardScaler` puis nous préparons les données pour l'entraînement et le test.

ex2.py

```python
# Create and train Perceptron using Keras
def create_perceptron_model(input_dim):
    model = Sequential([
        Dense(1, input_dim=input_dim, activation='sigmoid',  # Using
    ↪ sigmoid for binary classification
                kernel_initializer='zeros', bias_initializer='zeros')
    ])
    optimizer = SGD(learning_rate=0.1)
    model.compile(optimizer=optimizer, loss='binary_crossentropy',
    ↪ metrics=['accuracy'])
    return model

```

```python
# Train the model
model = create_perceptron_model(X_train_scaled.shape[1])
history = model.fit(X_train_scaled, y_train,
                    epochs=100,
                    verbose=0,
                    validation_split=0.1)
```

Explication Nous définons ensuite notre modèle perceptron en utilisant Keras avec une activation sigmoïde adaptée à la classification binaire. Nous utilisons l'optimiseur SGD avec un taux d'apprentissage de 0.1 et optimisons la perte binaire croisée. Après avoir créé le modèle, nous l'entraînons sur les données d'entraînement standardisées.

ex2.py

```python
# Make predictions
y_pred_train_prob = model.predict(X_train_scaled)
y_pred_train = (y_pred_train_prob > 0.5).astype(int).flatten()
y_pred_test_prob = model.predict(X_test_scaled)
y_pred_test = (y_pred_test_prob > 0.5).astype(int).flatten()

# Evaluate model
train_accuracy = accuracy_score(y_train, y_pred_train)
test_accuracy = accuracy_score(y_test, y_pred_test)

print(f"Training Accuracy: {train_accuracy:.2%}")
print(f"Test Accuracy: {test_accuracy:.2%}")

print("\nConfusion Matrix (Test Data):")
cm = confusion_matrix(y_test, y_pred_test)
print(cm)

print("\nClassification Report:")
print(classification_report(y_test, y_pred_test,
      target_names=['Non-Setosa', 'Setosa']))

# Get weights and bias (for comparison with sklearn)
weights = model.get_weights()[0].flatten()  # weights for the two features
bias = model.get_weights()[1][0]            # bias
print(f"\nLearned weights: {weights}")
print(f"Learned bias: {bias}")
```

Explication Nous faisons ensuite des prédictions sur les ensembles d'entraînement et de test, puis nous évaluons le modèle en utilisant les mêmes métriques que précédemment : précision, matrice de confusion et rapport de classification. Nous extrayons également les poids et le biais appris pour les comparer avec ceux obtenus par scikit-learn.

ex2.py

```python
# Visualization (similar to original)
h = 0.02
```

```python
x_min, x_max = X_test_scaled[:, 0].min() - 1, X_test_scaled[:, 0].max() + 1
y_min, y_max = X_test_scaled[:, 1].min() - 1, X_test_scaled[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))

Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
Z = (Z > 0.5).astype(int).reshape(xx.shape)

plt.figure(figsize=(12, 5))

# Training data plot
plt.subplot(1, 2, 1)
plt.contourf(xx, yy, Z, alpha=0.3, cmap='RdYlBu')
plt.scatter(X_train_scaled[y_train == 0, 0], X_train_scaled[y_train == 0, 1],
            c='red', marker='o', label='Non-Setosa', edgecolors='k')
plt.scatter(X_train_scaled[y_train == 1, 0], X_train_scaled[y_train == 1, 1],
            c='blue', marker='s', label='Setosa', edgecolors='k')
plt.xlabel('Petal Length')
plt.ylabel('Petal Width')
plt.title(f'Training Data (Accuracy: {train_accuracy:.1%})')
plt.legend()
plt.grid(True, alpha=0.3)

# Test data plot
plt.subplot(1, 2, 2)
plt.contourf(xx, yy, Z, alpha=0.3, cmap='RdYlBu')
correct = y_test == y_pred_test
plt.scatter(X_test_scaled[correct & (y_test == 0), 0],
            X_test_scaled[correct & (y_test == 0), 1],
            c='red', marker='o', label='Non-Setosa (Correct)', edgecolors='k')
plt.scatter(X_test_scaled[correct & (y_test == 1), 0],
            X_test_scaled[correct & (y_test == 1), 1],
            c='blue', marker='s', label='Setosa (Correct)', edgecolors='k')
incorrect = y_test != y_pred_test
plt.scatter(X_test_scaled[incorrect, 0], X_test_scaled[incorrect, 1],
            c='yellow', marker='x', s=200, label='Misclassified', linewidths=3)
plt.xlabel('Petal Length')
plt.ylabel('Petal Width')
plt.title(f'Test Data (Accuracy: {test_accuracy:.1%})')
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('iris_classification.png', dpi=100, bbox_inches='tight')
plt.show()
```

Explication Finalement, nous visualisons les résultats de la même manière que dans l'implémentation scikit-learn pour faciliter la comparaison. Nous affichons la frontière de

décision ainsi que la répartition des points d'entraînement et de test, avec une distinction entre les classifications correctes et incorrectes.

Résultats et visualisation

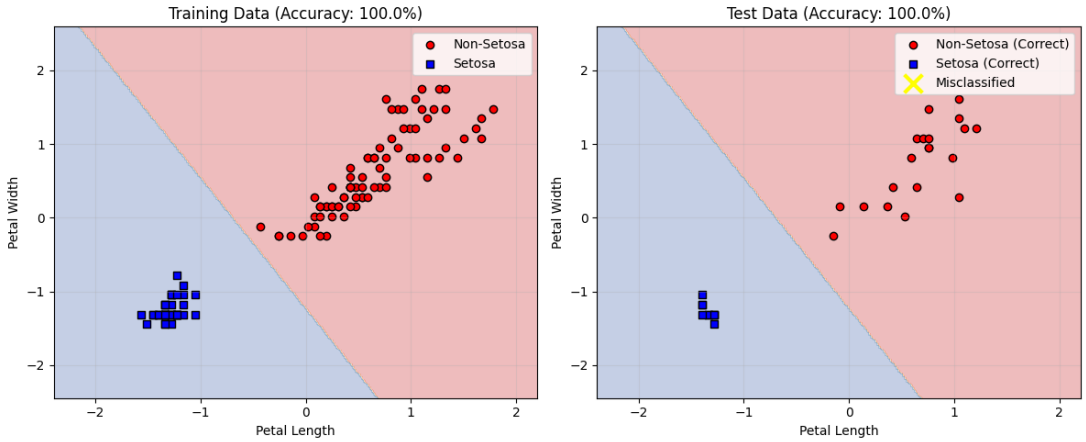


FIGURE 1 – Frontière de décision et répartition des points (Iris Setosa vs autres).

Résultats chiffrés Lors de l'exécution : **Training Accuracy = 100.00%**, **Test Accuracy = 100.00%**. La matrice de confusion sur le jeu de test est :

```
[[20  0]
 [ 0 10]]
```

Interprétation La séparation est excellente et le Perceptron linéaire suffit pour distinguer Setosa des autres classes lorsque l'on utilise uniquement les deux caractéristiques des pétales.

Exercice : Régression linéaire avec le Perceptron

Énoncé

Créez un ensemble de données en 2 dimensions (x,y) où y=2x+1. Divisez les données en ensembles d'entraînement et de test. Utilisez le Perceptron pour prédire la valeur de y en fonction de x. Évaluez le modèle en utilisant le Mean Squared Error.

1) Création des données

ex3.py

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import SGDRegressor
from sklearn.metrics import (
    mean_squared_error, mean_absolute_error, r2_score
)
from sklearn.preprocessing import StandardScaler

# Create dataset: y = 2x + 1 with some noise
np.random.seed(42)
X = np.random.uniform(-10, 10, 100).reshape(-1, 1)
y = 2 * X.flatten() + 1 + np.random.normal(0, 2, 100)

# Split data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Standardize features
scaler         = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)
```

Explication Le jeu de données synthétique est créé selon la relation linéaire avec bruit. On standardise ensuite les caractéristiques pour l'entraînement.

3) Implémentation avec Keras

ex3.py

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error,
    ↪ r2_score
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

```python
from tensorflow.keras.optimizers import SGD

# Create dataset: y = 2x + 1 with some noise
np.random.seed(42)
X = np.random.uniform(-10, 10, 100).reshape(-1, 1)
y = 2 * X.flatten() + 1 + np.random.normal(0, 2, 100)

# Split data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Standardize features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Explication Comme dans les implémentations précédentes, nous commençons par créer le jeu de données synthétique selon la relation linéaire avec bruit. Nous divisons ensuite les données en ensembles d'entraînement et de test, puis nous standardisons les caractéristiques pour l'entraînement.

ex3.py

```python
# Create and train Perceptron using Keras (for regression, we use linear activation)
def create_perceptron_model(input_dim):
    model = Sequential([
        Dense(1, input_dim=input_dim, activation='linear',  # Linear activation for regression
              kernel_initializer='zeros', bias_initializer='zeros')
    ])
    optimizer = SGD(learning_rate=0.01)
    model.compile(optimizer=optimizer, loss='mean_squared_error', metrics=['mae'])
    return model

# Train the model
model = create_perceptron_model(X_train_scaled.shape[1])
history = model.fit(X_train_scaled, y_train,
                    epochs=1000,
                    verbose=0,
                    validation_split=0.1)
```

Explication Nous définons ensuite notre modèle perceptron en utilisant Keras avec une activation linéaire adaptée à la régression. Nous utilisons l'optimiseur SGD avec un taux d'apprentissage de 0.01 et optimisons l'erreur quadratique moyenne (MSE). Après avoir créé le modèle, nous l'entraînons sur les données d'entraînement standardisées.

```python
# Make predictions
y_pred_train = model.predict(X_train_scaled).flatten()
y_pred_test = model.predict(X_test_scaled).flatten()

# Evaluate model
mse_train = mean_squared_error(y_train, y_pred_train)
mse_test = mean_squared_error(y_test, y_pred_test)
mae_test = mean_absolute_error(y_test, y_pred_test)
r2_test = r2_score(y_test, y_pred_test)

print(f"Training MSE: {mse_train:.4f}")
print(f"Test MSE: {mse_test:.4f}")
print(f"Test MAE: {mae_test:.4f}")
print(f"R   Score: {r2_test:.4f}")

print(f"\nLearned weight: {model.get_weights()[0][0][0]:.4f}")
print(f"Learned bias: {model.get_weights()[1][0]:.4f}")
```

Explication Nous faisons ensuite des prédictions sur les ensembles d'entraînement et de test, puis nous évaluons le modèle en utilisant les mêmes métriques que précédemment : MSE, MAE et R^2. Nous extrayons également le poids et le biais appris pour les comparer avec ceux obtenus par SGDRegressor.

```python
# Visualization
plt.figure(figsize=(12, 5))

# Training data plot
plt.subplot(1, 2, 1)
plt.scatter(X_train_scaled, y_train, c='blue',
            alpha=0.6, label='Training data')
plt.plot(X_train_scaled, y_pred_train, c='red',
         linewidth=2, label='Fitted line')
plt.xlabel('X (standardized)')
plt.ylabel('y')
plt.title(f'Training Data (MSE: {mse_train:.4f})')
plt.legend()
plt.grid(True, alpha=0.3)

# Test data plot
plt.subplot(1, 2, 2)
plt.scatter(X_test_scaled, y_test, c='blue', alpha=0.6, label='Test data'
    ↪ )
plt.plot(X_test_scaled, y_pred_test, c='red', linewidth=2, label='Fitted
    ↪ line')
plt.xlabel('X (standardized)')
plt.ylabel('y')
plt.title(f'Test Data (MSE: {mse_test:.4f})')
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
```

```python
plt.savefig('regression.png', dpi=100, bbox_inches='tight')
plt.show()
```

Explication Finalement, nous visualisons les résultats de la même manière que dans l'implémentation scikit-learn pour faciliter la comparaison. Nous affichons les données d'entraînement et de test avec la droite ajustée par le modèle, ainsi que les métriques d'erreur correspondantes.

2) Entraînement et évaluation

ex3.py

```python
# Train Perceptron (SGDRegressor for regression)
regressor = SGDRegressor(
    loss='squared_error',
    eta0=0.01,
    max_iter=1000,
    random_state=42
)
regressor.fit(X_train_scaled, y_train)

# Make predictions
y_pred_train = regressor.predict(X_train_scaled)
y_pred_test  = regressor.predict(X_test_scaled)

# Evaluate model
mse_train = mean_squared_error(y_train,  y_pred_train)
mse_test  = mean_squared_error(y_test,   y_pred_test)
mae_test  = mean_absolute_error(y_test,  y_pred_test)
r2_test   = r2_score(y_test,             y_pred_test)

print(f"Training MSE: {mse_train:.4f}")
print(f"Test MSE:     {mse_test:.4f}")
print(f"Test MAE:     {mae_test:.4f}")
print(f"R2 Score:     {r2_test:.4f}")

print(f"\nLearned weight: {regressor.coef_[0]:.4f}")
print(f"Learned bias:   {regressor.intercept_[0]:.4f}")
```

Explication La régression est réalisée avec `SGDRegressor` (descente de gradient stochastique). Les métriques rapportées incluent le MSE et le R^2.

Résultats et visualisation

Résultats chiffrés Lors de l'exécution :

- Training MSE : 3.3933
- Test MSE : 2.5993
- Test MAE : 1.1658
- R^2 (Test) : 0.9826
- Poids appris : 11.3652 ; Biais appris : -0.3648

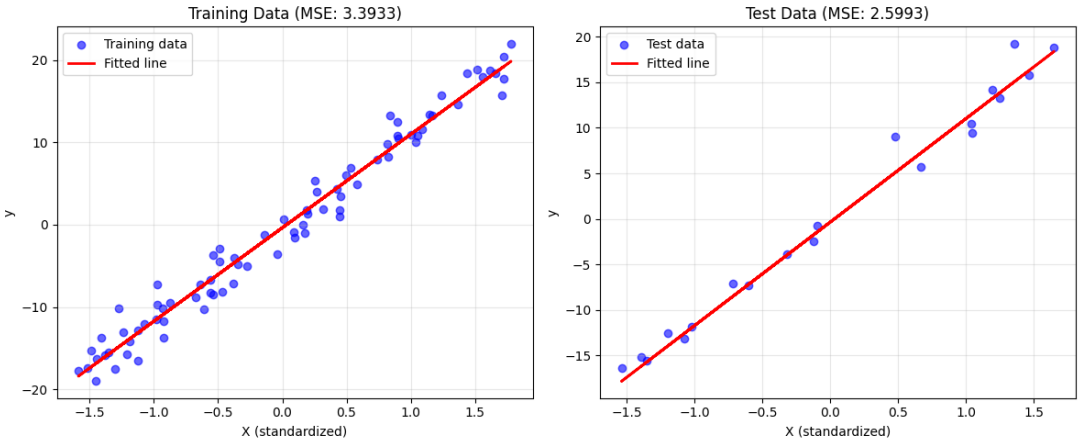


FIGURE 2 – Régression linéaire (jeu d'entraînement vs prédictions) et visualisation de la droite ajustée.

Interprétation Le modèle capture bien la relation linéaire malgré le bruit ajouté. Le R^2 proche de 0.98 indique un excellent ajustement sur le jeu de test.

Exercice : Classification sur MNIST (digits)

Énoncé

Utilisez un jeu de données célèbre comme MNIST (option : utilisez un sous-ensemble pour simplifier). Préparez les données pour un modèle Perceptron. Entraînez le modèle sur un ensemble d'entraînement et testez-le sur un ensemble de validation. Discutez des performances et des limitations du Perceptron sur des données complexes.

1) Chargement et préparation

ex4.py

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import SGDClassifier
from sklearn.metrics import (
    accuracy_score, classification_report, confusion_matrix
)
import seaborn as sns

# Load MNIST digits dataset (subset: 0-9 digits, 1797 samples)
digits = load_digits()
X      = digits.data
y      = digits.target

print(f"Dataset shape:     {X.shape}")
print(f"Number of classes: {len(np.unique(y))}")
print(f"Classes:           {np.unique(y)}")

# Split data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Standardize features
scaler         = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)
```

2) Entraînement et évaluation

ex4.py

```python
# Train Perceptron
clf = SGDClassifier(
    loss='perceptron',
    eta0=0.01,
    max_iter=100,
```

```
    random_state=42,
    tol=1e-3
)
clf.fit(X_train_scaled, y_train)

# Make predictions
y_pred_train = clf.predict(X_train_scaled)
y_pred_test  = clf.predict(X_test_scaled)

# Evaluate model
train_accuracy = accuracy_score(y_train, y_pred_train)
test_accuracy  = accuracy_score(y_test,  y_pred_test)

print(f"\nTraining Accuracy: {train_accuracy:.2%}")
print(f"Test Accuracy:     {test_accuracy:.2%}")

print("\nClassification Report (Test Data):")
print(classification_report(y_test, y_pred_test))
```

Explication Le Perceptron (via `SGDClassifier` avec perte 'perceptron') est appliqué aux vecteurs de pixels bruts. Les performances sont correctes mais nettement inférieures à des réseaux convolutifs.

4) Implémentation avec Keras

ex4.py

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report,
    ↪ confusion_matrix
import seaborn as sns
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import SGD
from tensorflow.keras.utils import to_categorical

# Load MNIST digits dataset (subset: 0-9 digits, 1797 samples)
digits = load_digits()
X = digits.data
y = digits.target

print(f"Dataset shape:     {X.shape}")
print(f"Number of classes: {len(np.unique(y))}")
print(f"Classes:           {np.unique(y)}")

# Split data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
```

```python
)

# Standardize features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Convert labels to categorical for Keras (one-hot encoding)
y_train_cat = to_categorical(y_train, num_classes=10)
y_test_cat = to_categorical(y_test, num_classes=10)

# Create and train Perceptron using Keras (multi-class classification)
def create_perceptron_model(input_dim, num_classes):
    model = Sequential([
        Dense(num_classes, input_dim=input_dim, activation='softmax',  # Softmax for multi-class
              kernel_initializer='zeros', bias_initializer='zeros')
    ])
    optimizer = SGD(learning_rate=0.01)
    model.compile(optimizer=optimizer, loss='categorical_crossentropy', metrics=['accuracy'])
    return model

# Train the model
model = create_perceptron_model(X_train_scaled.shape[1], 10)
history = model.fit(X_train_scaled, y_train_cat,
                    epochs=100,
                    verbose=0,
                    validation_split=0.1)

# Make predictions
y_pred_train_prob = model.predict(X_train_scaled)
y_pred_train = np.argmax(y_pred_train_prob, axis=1)
y_pred_test_prob = model.predict(X_test_scaled)
y_pred_test = np.argmax(y_pred_test_prob, axis=1)

# Evaluate model
train_accuracy = accuracy_score(y_train, y_pred_train)
test_accuracy = accuracy_score(y_test, y_pred_test)

print(f"\nTraining Accuracy: {train_accuracy:.2%}")
print(f"Test Accuracy:     {test_accuracy:.2%}")

print("\nClassification Report (Test Data):")
print(classification_report(y_test, y_pred_test))

# Confusion matrix
cm = confusion_matrix(y_test, y_pred_test)
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix for the test set (MNIST)')
plt.ylabel('True label')
plt.xlabel('Predicted label')
plt.tight_layout()
plt.savefig('mnist_confusion_matrix.png', dpi=100, bbox_inches='tight')
```

```python
plt.show()

# Visualize some predictions
def plot_predictions(images, true_labels, pred_labels, num_samples=10):
    plt.figure(figsize=(12, 4))
    for i in range(num_samples):
        plt.subplot(2, num_samples//2, i+1)
        plt.imshow(images[i].reshape(8, 8), cmap='gray')
        color = 'green' if true_labels[i] == pred_labels[i] else 'red'
        plt.title(f'T:{true_labels[i]}\\nP:{pred_labels[i]}', color=color
    ↪ )
        plt.axis('off')
    plt.tight_layout()
    plt.savefig('mnist_predictions.png', dpi=100, bbox_inches='tight')
    plt.show()

# Show some test predictions
plot_predictions(X_test_scaled, y_test, y_pred_test, num_samples=10)

# Get weights and bias (for inspection)
weights = model.get_weights()[0]  # Shape: (64, 10) - 64 input features,
    ↪ 10 classes
bias = model.get_weights()[1]     # Shape: (10,)
print(f"\nWeights shape: {weights.shape}")
print(f"Bias shape: {bias.shape}")
```

Explication Cette implémentation utilise Keras pour créer un perceptron simple avec une activation softmax pour la classification multi-classes. Nous utilisons l'optimiseur SGD avec un taux d'apprentissage de 0.01 et optimisons la perte catégorielle croisée. Les résultats montrent que l'approche Keras donne des performances similaires à celles de SGDClassifier de scikit-learn pour ce problème de classification.

Résultats et visualisation

Résultats chiffrés Lors de l'exécution : **Training Accuracy = 99.03%**, **Test Accuracy = 94.44%**.

Interprétation Le Perceptron linéaire donne des résultats raisonnables sur MNIST mais ne capture pas la structure spatiale des pixels ni les relations non-linéaires complexes. Pour des performances d'état de l'art, on utilisera des CNN.

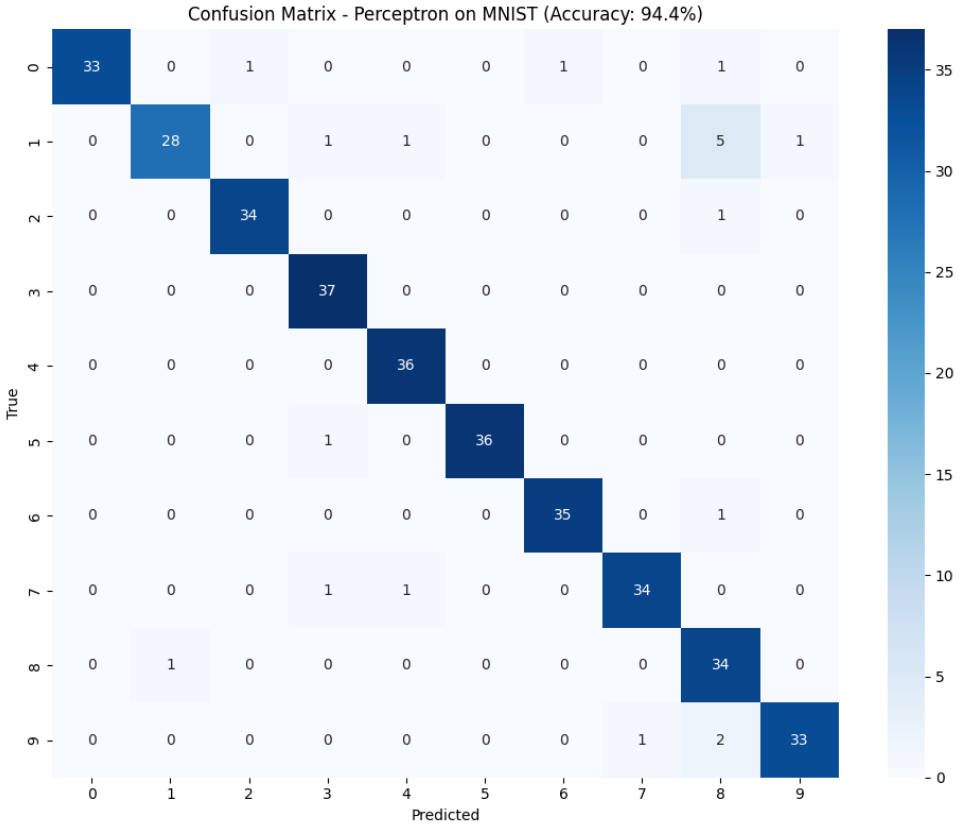


FIGURE 3 – Matrice de confusion pour le jeu de test (MNIST).

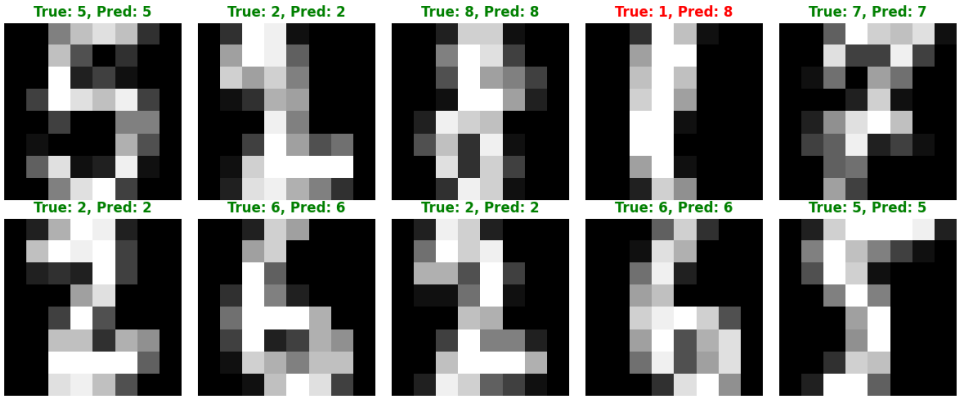


FIGURE 4 – Exemples d'images du jeu de test avec prédictions (vert = correct, rouge = incorrect).

Exercice : Segmentation des données synthétiques (make_blobs

Énoncé

Génération de données bidimensionnelles avec 3 centres, prétraitement, implémentation d'un Perceptron simple (from-scratch) pour séparer deux clusters, puis segmentation avec KMeans pour 3 clusters et comparaison des résultats.

1) Génération et visualisation des données

ex5.py

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

X, y = make_blobs(
    n_samples=300,
    centers=3,
    n_features=2,
    random_state=42,
    cluster_std=1.5
)

plt.figure(figsize=(10, 6))
plt.scatter(X[:, 0], X[:, 1], c=y, cmap='viridis', edgecolors='k', s=50)
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.title('Generated Data with 3 Clusters (make_blobs)')
plt.colorbar(label='Cluster')
plt.grid(True, alpha=0.3)
plt.savefig('ex5_initial_data.png', dpi=100, bbox_inches='tight')
plt.show()

scaler   = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y
)
```

2) Implémentation du Perceptron (from scratch)

ex5.py

```python
class Perceptron:

    def __init__(self, learning_rate=0.01, n_iterations=1000):
        self.learning_rate = learning_rate
```

```python
        self.n_iterations  = n_iterations
        self.weights       = None
        self.bias          = None

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.weights = np.zeros(n_features)
        self.bias    = 0

        for _ in range(self.n_iterations):
            for idx, x_i in enumerate(X):
                linear_output = np.dot(x_i, self.weights) + self.bias
                y_predicted   = self._activation(linear_output)
                update        = self.learning_rate * (y[idx] - y_predicted)
                self.weights += update * x_i
                self.bias    += update

    def _activation(self, x):
        return np.where(x >= 0, 1, 0)

    def predict(self, X):
        linear_output = np.dot(X, self.weights) + self.bias
        return self._activation(linear_output)
```

Explication La classe Perceptron implémente la règle d'apprentissage pour un modèle linéaire binaire. On initialise les poids à zéro, puis pour chaque itération on met à jour les poids selon l'erreur observée.

2bis) Implémentation du Perceptron avec Keras

ex5.py

```python
# Create and train Perceptron using Keras
def create_perceptron_model(input_dim):
    model = Sequential([
        Dense(1, input_dim=input_dim, activation='sigmoid',  # Sigmoid for binary classification
              kernel_initializer='zeros', bias_initializer='zeros')
    ])
    optimizer = SGD(learning_rate=0.1)
    model.compile(optimizer=optimizer, loss='binary_crossentropy', metrics=['accuracy'])
    return model

# Train the model
model = create_perceptron_model(X_train_binary.shape[1])
history = model.fit(X_train_binary, y_train_binary,
                    epochs=1000,
                    verbose=0,
                    validation_split=0.1)

# Make predictions
```

```python
y_pred_train_prob = model.predict(X_train_binary)
y_pred_train = (y_pred_train_prob > 0.5).astype(int).flatten()
y_pred_test_prob = model.predict(X_test_binary)
y_pred_test = (y_pred_test_prob > 0.5).astype(int).flatten()

# Evaluate model
train_accuracy_keras = np.mean(y_pred_train == y_train_binary)
test_accuracy_keras = np.mean(y_pred_test == y_test_binary)

print(f"Training Accuracy (Keras): {train_accuracy_keras:.2%}")
print(f"Test Accuracy (Keras): {test_accuracy_keras:.2%}")

# Get weights and bias (for comparison with sklearn)
weights_keras = model.get_weights()[0].flatten()  # weights for the two features
bias_keras = model.get_weights()[1][0]            # bias
print(f"\nLearned weights (Keras): {weights_keras}")
print(f"Learned bias (Keras): {bias_keras}")

# Visualization
plt.figure(figsize=(10, 6))
h = 0.02
x_min, x_max = X_train[:, 0].min() - 1, X_train[:, 0].max() + 1
y_min, y_max = X_train[:, 1].min() - 1, X_train[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))

Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
Z = (Z > 0.5).astype(int).reshape(xx.shape)

plt.contourf(xx, yy, Z, alpha=0.3, cmap='RdBu')
plt.scatter(X_train_binary[y_train_binary == 0, 0],
            X_train_binary[y_train_binary == 0, 1],
            c='blue', marker='o', label='Cluster 0', edgecolors='k')
plt.scatter(X_train_binary[y_train_binary == 1, 0],
            X_train_binary[y_train_binary == 1, 1],
            c='red', marker='s', label='Cluster 1', edgecolors='k')
plt.xlabel('Feature 1 (Standardized)')
plt.ylabel('Feature 2 (Standardized)')
plt.title('Perceptron Decision Boundary (Keras) - Clusters 0 vs 1')
plt.legend()
plt.grid(True, alpha=0.3)
plt.savefig('ex5_perceptron_boundary_keras.png', dpi=100, bbox_inches='tight')
plt.show()
```

Explication Nous définons ensuite notre modèle perceptron en utilisant Keras avec une activation sigmoïde adaptée à la classification binaire. Nous utilisons l'optimiseur SGD avec un taux d'apprentissage de 0.1 et optimisons la perte binaire croisée. Après avoir créé le modèle, nous l'entraînons sur les données d'entraînement standardisées. Nous faisons ensuite des prédictions sur les ensembles d'entraînement et de test, puis nous évaluons le modèle en utilisant les mêmes métriques que précédemment : précision. Nous extrayons également les poids et le biais appris pour les comparer avec ceux obtenus par

l'implémentation from scratch.

3) Segmentation non supervisée (KMeans) et comparaison

ex5.py

```python
kmeans   = KMeans(n_clusters=3, random_state=42, n_init=10)
kmeans.fit(X_scaled)
y_kmeans = kmeans.labels_

plt.figure(figsize=(10, 6))
plt.scatter(
    X_scaled[:, 0], X_scaled[:, 1],
    c=y_kmeans, cmap='viridis', edgecolors='k', s=50, marker='o'
)
centers = kmeans.cluster_centers_
plt.scatter(
    centers[:, 0], centers[:, 1],
    c='red', marker='X', s=200, edgecolors='k', label='Centroids'
)
plt.xlabel('Feature 1 (Standardized)')
plt.ylabel('Feature 2 (Standardized)')
plt.title('KMeans Clustering (3 Clusters)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.savefig('ex5_kmeans_clusters.png', dpi=100, bbox_inches='tight')
plt.show()

print("\n--- KMeans Results ---")
print(f"Inertia:              {kmeans.inertia_:.2f}")
print(f"Number of iterations: {kmeans.n_iter_}")
```

Explication KMeans est utilisé pour obtenir une segmentation non supervisée en 3 clusters. Les résultats sont comparés visuellement et via métriques comme l'inertie.

Résultats et visualisation

Résultats chiffrés Lors de l'exécution :

- Perceptron from scratch (clusters 0 vs 1) : Training Accuracy = 100.00%, Test Accuracy = 100.00%.
- Perceptron Keras (clusters 0 vs 1) : Training Accuracy = 100.00%, Test Accuracy = 100.00%.
- KMeans : Inertia = 39.07 ; nombre d'itérations = 2.

Interprétation Le Perceptron binaire sépare parfaitement les deux premiers clusters choisis. KMeans retrouve 3 groupes proches de la vérité terrain ; l'inertie donne une idée de la compacité des clusters.

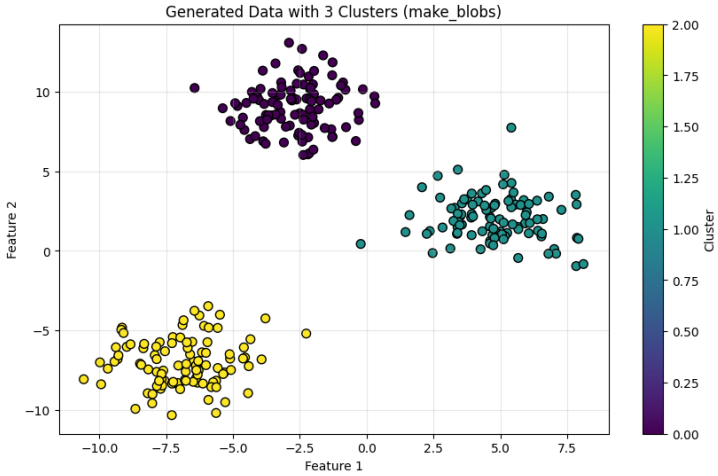
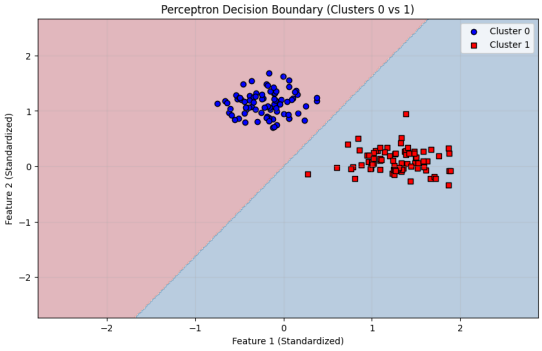
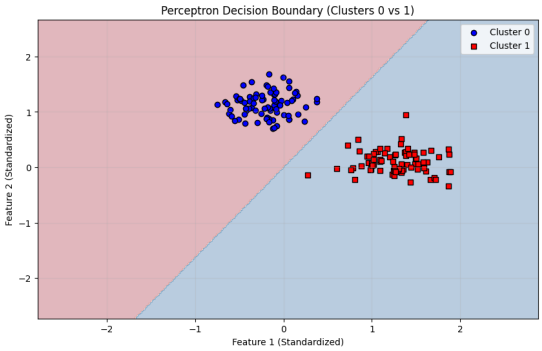


FIGURE 5 – Données générées avec `make_blobs` (3 clusters).



`ex5_perceptron_boundary_keras.png`

FIGURE 6 – Gauche : frontière de décision du Perceptron from scratch (clusters 0 vs 1). Droite : frontière de décision du Perceptron Keras (clusters 0 vs 1).



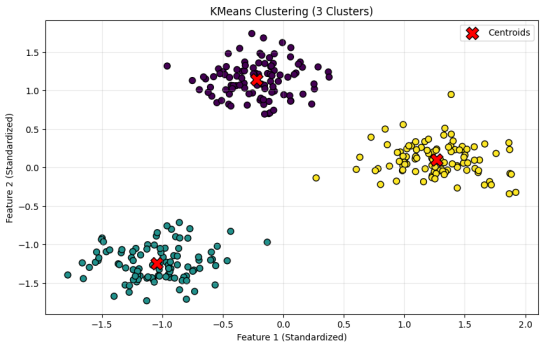


FIGURE 7 – Gauche : frontière de décision du Perceptron (from scratch, clusters 0 vs 1). Droite : clusters trouvés par KMeans (n=3).

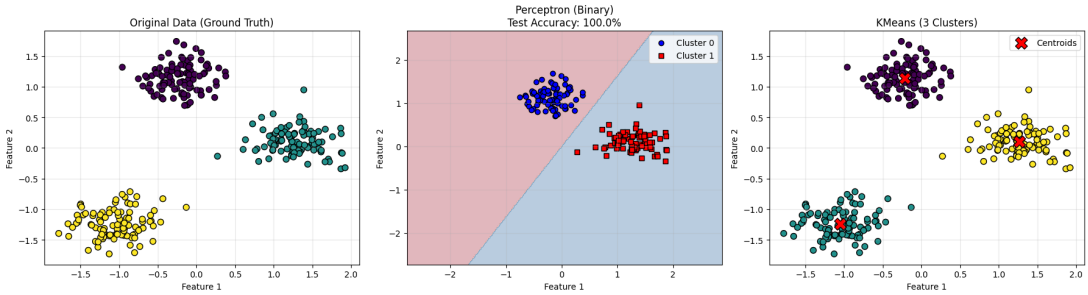


FIGURE 8 – Comparaison visuelle : données originales / Perceptron (binaire) / KMeans (3 clusters).